

SIMATS ENGINEERING – COMPUTER ARCHITECTURE (CSA12)

ASSESSMENT TOOL 3 – CO1 ANSWERS

1. CPU Instruction Cycle – Fetch, Decode, Execute and Store

The following Python program simulates the basic instruction cycle of a CPU. It reads an ADD instruction, identifies the operation and operands, performs the arithmetic operation, and represents storing the result in a register.

```
instruction = input().split()
operation = instruction[0]
operand1 = int(instruction[1])
operand2 = int(instruction[2])

print("Fetched Instruction")

print("Decoded as", operation)

if operation == "ADD":
    result = operand1 + operand2
elif operation == "SUB":
    result = operand1 - operand2
else:
    result = 0

print("Executed Result =", result)
print("Stored in Register")
```

Sample Input: ADD 15 20

Output: Fetched Instruction | Decoded as ADD | Executed Result = 35 | Stored in Register

2. Counting Memory Accesses

LOAD is treated as a memory read because data is transferred from memory to the CPU. STORE is treated as a memory write because data is transferred from the CPU to memory. ADD and SUB do not directly access memory in this given sequence.

```
instructions = ["LOAD", "ADD", "STORE", "SUB", "LOAD", "STORE"]

memory_reads = 0
memory_writes = 0

for instruction in instructions:
    if instruction == "LOAD":
        memory_reads += 1
    elif instruction == "STORE":
        memory_writes += 1

print("Memory Reads =", memory_reads)
print("Memory Writes =", memory_writes)
```

For LOAD, ADD, STORE, SUB, LOAD, STORE: Memory Reads = 2 and Memory Writes = 2.

3. Five Stages of CPU Execution Cycle

The five stages are Fetch, Decode, Execute, Memory Access and Write Back. The program below displays all five stages for every instruction in the given sequence.

```
instructions = ["LOAD R1", "ADD R2", "STORE R3"]
stages = ["Fetch", "Decode", "Execute",
          "Memory Access", "Write Back"]

for instruction in instructions:
    print("Instruction:", instruction)
    for stage in stages:
        print(stage)
```

The same five stages are displayed for LOAD R1, ADD R2 and STORE R3, as required in the question.

4. Exchange the Contents of Registers B and C

To exchange two register values without losing either value, a temporary variable is used. First, the value of B is copied to TEMP. Then C is copied to B, and finally TEMP is copied to C.

```
B = 0x25  
C = 0x60
```

```
TEMP = B  
B = C  
C = TEMP
```

```
print("B =", hex(B).upper())  
print("C =", hex(C).upper())
```

Before exchange: B = 25H, C = 60H

After exchange: B = 60H, C = 25H

5(i). Add Two 8-bit Numbers and Store the Result

The numbers are stored in memory locations 2500H and 2501H. The first number is loaded into AL, the second number into BL, and ADD performs the addition. The result is stored at memory location 2502H.

```
MOV SI, 2500H  
MOV AL, [SI]    ; Load number from 2500H  
MOV BL, [SI+1]  ; Load number from 2501H  
ADD AL, BL      ; Add the two numbers  
MOV [SI+2], AL  ; Store result at 2502H  
HLT
```

5(ii). Subtract Two 8-bit Numbers and Store the Result

The number at 2501H is subtracted from the number at 2500H. The result of the subtraction is stored at 2502H.

```
MOV SI, 2500H  
MOV AL, [SI]    ; Load number from 2500H  
MOV BL, [SI+1]  ; Load number from 2501H  
SUB AL, BL      ; 2500H value - 2501H value  
MOV [SI+2], AL  ; Store result at 2502H  
HLT
```

Answer Summary

1. CPU cycle: Fetch → Decode → Execute → Store. 2. Memory accesses: 2 reads and 2 writes. 3. Five stages: Fetch → Decode → Execute → Memory Access → Write Back. 4. Register exchange: B = 60H and C = 25H after exchange. 5. The 8086 programs perform the required addition and subtraction and store the results at 2502H.